



REGISTRO DE DECISÃO ARQUITETURAL

Decisões de persistência e consistência de dados

Justificativa técnica para PostgreSQL, MongoDB, Redis e OpenSearch, com os limites de responsabilidade de cada tecnologia na plataforma.

Persistência poliglota com responsabilidade explícita

A plataforma combina dados transacionais, consultas públicas de alta frequência, busca textual e auditoria. Uma única tecnologia conseguiria armazenar todos esses dados, mas imporia compromissos inadequados de consistência, custo de consulta ou evolução de esquema.

Princípio: PostgreSQL é a fonte de verdade transacional. Redis e OpenSearch são projeções reconstruíveis. MongoDB preserva a trilha de auditoria append-only.

Critérios usados

Consistência

Campanhas, doações e pagamentos precisam de transações ACID, restrições e idempotência verificável.

Padrão de acesso

Listagens públicas, busca fuzzy e consulta cronológica de eventos têm perfis de leitura distintos.

Recuperação

Cache e índices devem ser reconstruíveis sem perda do registro financeiro ou da auditoria.

Operação

Cada componente expõe health checks, métricas e configuração externa para execução local ou gerenciada.

Tecnologia	Responsabilidade	Consistência	Fonte de verdade
PostgreSQL	Campanhas, perfis, doações, pagamentos, webhooks, Outbox e Inbox.	Forte / transacional	Sim
MongoDB	Eventos de auditoria por tenant e correlação.	Append-only por documento	Sim, para auditoria
Redis	Campanhas ativas e respostas públicas frequentes.	Eventual / TTL	Não
OpenSearch	Busca fuzzy de campanhas e recuperação de fontes documentais.	Eventual / indexada	Não

PostgreSQL para o núcleo; MongoDB para auditoria

PostgreSQL

O domínio financeiro exige atomicidade entre estado e mensagens. Campanhas, doações e pagamentos possuem relações, regras de unicidade e mudanças de estado que se beneficiam de constraints e transações. O EF Core mantém o modelo explícito e permite testes relacionais com SQLite sem substituir a validação em PostgreSQL no runtime.

- Índices compostos por `TenantId` e estado suportam isolamento lógico e consultas operacionais.
- Outbox e Inbox persistem eventos e consumo idempotente junto da alteração de negócio.
- Webhooks de pagamento possuem chave do provedor e não confirmam a mesma transação duas vezes.
- O ledger permanece íntegro mesmo quando Redis, OpenSearch ou RabbitMQ estão temporariamente indisponíveis.

MongoDB

A auditoria recebe envelopes de eventos heterogêneos. O documento preserva o payload original, enquanto campos estáveis - tenant, tipo, origem, correlação e instante - permanecem indexados. Esse formato evita alterar uma tabela relacional a cada evolução de contrato e mantém a consulta cronológica simples.

- Coleção append-only; a API expõe leitura, não mutação de registros.
- Índices (`TenantId`, `Timestamp`) e (`TenantId`, `CorrelationId`).
- Payload serializado para rastreabilidade sem acoplar o armazenamento aos assemblies dos produtores.

Trade-off aceito: uma consulta analítica que combine ledger e auditoria atravessa duas fontes. A correlação é feita por `TenantId`, `CorrelationId`, `EventId` e identificadores de agregado, não por transação distribuída.

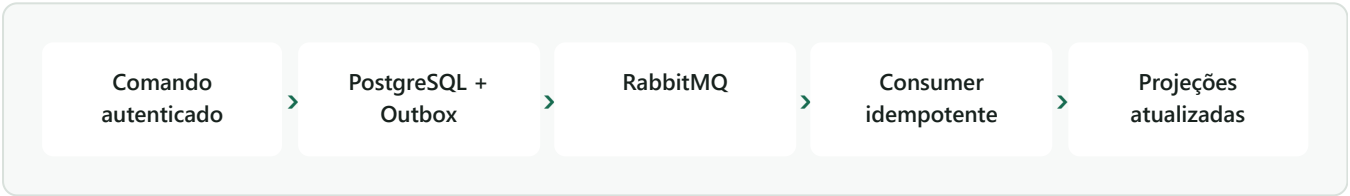
Redis e OpenSearch aceleram leitura sem controlar o domínio

Redis

O painel público concentra leituras repetidas de campanhas ativas. Redis reduz latência e carga no banco relacional, com chave segmentada por tenant e expiração definida. A API sinaliza `X-Cache: MISS` ou `HIT`, tornando o comportamento observável.

OpenSearch

A busca precisa tolerar pequenas diferenças de escrita e pesquisar título e descrição. OpenSearch oferece análise textual e fuzzy matching sem degradar o modelo transacional. A mesma infraestrutura mantém documentos verificados usados pela Knowledge API, sempre com fontes retornadas ao usuário.



Modelo de consistência

Situação	Comportamento	Recuperação
Redis indisponível	Leitura recorre ao PostgreSQL; escrita do domínio continua.	Cache volta a ser preenchido sob demanda.
OpenSearch atrasado	CRUD e listagem transacional continuam; busca pode refletir estado anterior.	Reindexação a partir do PostgreSQL.
RabbitMQ indisponível	Evento permanece na Outbox até nova tentativa.	Publicação posterior sem duplicar efeito.
Consumer reprocessado	Inbox e chaves de idempotência rejeitam efeito duplicado.	Retry seguro e DLQ para intervenção.

Controles aplicados e caminho de evolução

Isolamento lógico

- `tenant_id` emitido pelo Keycloak e propagado no JWT.
- `TenantId` obrigatório em tabelas, documentos, eventos, cache e logs.
- Tenant autenticado prevalece sobre parâmetros enviados pelo cliente.

Segredos

- Nenhuma senha é versionada.
- Ambiente local gera valores criptograficamente aleatórios em `.env`.
- Kubernetes consome um Secret existente; nuvens usam secrets do ambiente de entrega.

Conclusão

A separação não existe para multiplicar tecnologias. Ela protege o ledger transacional, preserva auditoria independente e permite otimizar as leituras públicas. O desenho mantém uma origem autoritativa clara e define como cada projeção pode ser reconstruída.

Proteção de dados

- Senhas pertencem ao Keycloak e não ao domínio.
- CPF normalizado e protegido no perfil; logs não registram credenciais.
- Auditoria registra contexto técnico sem se tornar fonte de autenticação.

Produção

- PostgreSQL gerenciado com backup point-in-time e alta disponibilidade.
- Redis e OpenSearch gerenciados quando a escala justificar.
- Políticas de retenção e arquivamento para auditoria.

Implementação correspondente: PostgreSQL 17, MongoDB 8, Redis 8, OpenSearch 3, EF Core, MassTransit Outbox/Inbox e contratos de integração multi-tenant.